Checkpointing System API 문서

개요

Checkpointing System은 운영체제의 프로세스 체크포인팅(Checkpointing) 기법을 AI 에이전트의 장애 복구에 적용한 시스템입니다. 에이전트 상태 저장, 복구, 증분 백업을 통해 안정적인 AI 서비스 운영을 지원합니다.

기본 URL: `http://localhost:3002/api`

---

1. 시스템 상태 (System Status)

1.1 헬스 체크

```
GET /api/health
```

응답 예시:

```
{
  "status": "ok",
  "timestamp": "2025-01-25T10:30:00.000Z",
  "services": {
    "mongodb": "healthy",
    "api": "healthy"
  },
  "config": {
    "maxCheckpointsPerAgent": 10,
    "checkpointInterval": 30000,
    "maxStateSize": 52428800
  }
}
```

---

2. 체크포인트 관리 (Checkpoint Management)

2.1 체크포인트 생성

에이전트의 현재 상태를 저장합니다.

```
POST /api/checkpoints
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "123e4567-e89b-12d3-a456-426614174000",
  "state": {
    "messages": [
      {
        "role": "user",
        "content": "Hello",
        "timestamp": "2026-01-25T10:30:00.000Z"
      },
      {
        "role": "assistant",
        "content": "Hi there!",
        "timestamp": "2026-01-25T10:30:01.000Z"
      }
    ],
```

```
    "variables": {
      "counter": 5,
      "userName": "Alice"
    },
    "executionPosition": {
      "step": 3,
      "functionName": "processData"
    },
    "status": "running"
  },
  "options": {
    "type": "full",
    "description": "Processing step 3",
    "tags": ["important", "milestone"],
    "ttl": 3600
  }
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |——|——|——|——| | agentId | string | Y | 에이전트 UUID | | state | object | Y | 에이전트 상태 (JSON 직렬화 가능) | | state.messages | array | N | 메시지 히스토리 | | state.variables | object | N | 변수 상태 | | state.executionPosition | object | N | 실행 위치 정보 | | state.status | string | N | 에이전트 상태 (running, paused, etc.) | | options.type | string | N | 체크포인트 유형 (full, incremental) | | options.description | string | N | 체크포인트 설명 | | options.tags | array | N | 태그 목록 | | options.ttl | number | N | 생존 시간 (초), 기본값: 무제한 |

응답 예시:

```
{
  "success": true,
  "checkpoint": {
    "checkpointId": "ckpt-abc123",
    "agentId": "123e4567-e89b-12d3-a456-426614174000",
    "timestamp": "2026-01-25T10:30:00.000Z",
    "type": "full",
    "size": 1024,
    "description": "Processing step 3",
    "tags": ["important", "milestone"]
  }
}
```

2.2 체크포인트 목록 조회

특정 에이전트의 모든 체크포인트를 조회합니다.

```
GET /api/checkpoints/:agentId
```

응답 예시:

```
{
  "success": true,
  "checkpoints": [
    {
      "checkpointId": "ckpt-abc123",
      "timestamp": "2026-01-25T10:30:00.000Z",
      "type": "full",
      "size": 1024,
      "description": "Processing step 3"
    },
```

```
    {
      "checkpointId": "ckpt-def456",
      "timestamp": "2026-01-25T10:25:00.000Z",
      "type": "incremental",
      "size": 256,
      "description": "Processing step 2"
    }
  ],
  "total": 2
}
```

2.3 최신 체크포인트 조회

```
GET /api/checkpoints/:agentId/latest
```

응답 예시:

```
{
  "success": true,
  "checkpoint": {
    "checkpointId": "ckpt-abc123",
    "timestamp": "2026-01-25T10:30:00.000Z",
    "type": "full",
    "state": {
      "messages": [...],
      "variables": {...},
      "status": "running"
    }
  }
}
```

2.4 체크포인트 통계

```
GET /api/checkpoints/:agentId/stats
```

응답 예시:

```
{
  "success": true,
  "stats": {
    "totalCheckpoints": 15,
    "fullCheckpoints": 5,
    "incrementalCheckpoints": 10,
    "totalSize": 524288,
    "oldestCheckpoint": "2026-01-25T08:00:00.000Z",
    "newestCheckpoint": "2026-01-25T10:30:00.000Z",
    "averageSize": 34952
  }
}
```

---

3. 복구 (Recovery)

3.1 복구 실행

저장된 체크포인트에서 에이전트 상태를 복원합니다.

```
POST /api/checkpoints/recover
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "123e4567-e89b-12d3-a456-426614174000",
  "checkpointId": "ckpt-abc123",
  "verifyIntegrity": true,
  "fallbackToLatest": true
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |——|——|——|——| | agentId | string | Y | 에이전트 UUID | | checkpointId | string | N | 특정 체크포인트 ID (null이면 최신) | | verifyIntegrity | boolean | N | 무결성 검증 (기본값: true) | | fallbackToLatest | boolean | N | 실패 시 최신 체크포인트 시도 (기본값: true) |

응답 예시:

```
{
  "success": true,
  "restoredState": {
    "messages": [
      {
        "role": "user",
        "content": "Hello",
        "timestamp": "2026-01-25T10:30:00.000Z"
      },
      {
        "role": "assistant",
        "content": "Hi there!",
        "timestamp": "2026-01-25T10:30:01.000Z"
      }
    ],
    "variables": {
      "counter": 5,
      "userName": "Alice"
    },
    "executionPosition": {
      "step": 3,
      "functionName": "processData"
    },
    "status": "paused"
  },
  "recoveryTime": 15,
  "checkpointUsed": "ckpt-abc123",
  "integrityVerified": true
}
```

3.2 복구 실패 응답

```
{
  "success": false,
  "error": {
    "code": "CHECKPOINT_CORRUPTED",
    "message": "Checkpoint data is corrupted",
    "details": {
      "checkpointId": "ckpt-abc123",
      "issue": "Integrity check failed"
    }
  }
}
```

---

4. 체크포인트 삭제 (Deletion)

4.1 특정 체크포인트 삭제

```
DELETE /api/checkpoints/:checkpointId
```

응답 예시:

```
{
  "success": true,
  "message": "Checkpoint deleted successfully"
}
```

4.2 에이전트의 모든 체크포인트 삭제

```
DELETE /api/checkpoints/:agentId/all
```

응답 예시:

```
{
  "success": true,
  "deletedCount": 15
}
```

---

5. 주기적 체크포인트 (Periodic Checkpointing)

5.1 주기적 체크포인트 시작

자동으로 주기적으로 체크포인트를 생성합니다.

```
POST /api/checkpoints/periodic/start
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "123e4567-e89b-12d3-a456-426614174000",
  "intervalMs": 30000,
  "idleCheckpointsEnabled": true,
  "adaptiveInterval": true
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |——|——|——|——| | agentId | string | Y | 에이전트 UUID | | intervalMs | number | N | 체크포인트 간격 (밀리초, 기본값: 30000) | | idleCheckpointsEnabled | boolean | N | 유휴 상태에서도 체크포인트 생성 | | adaptiveInterval | boolean | N | 상태 변경량에 따라 간격 조정 |

응답 예시:

```
{
  "success": true,
  "message": "Periodic checkpointing started",
  "config": {
    "agentId": "123e4567-e89b-12d3-a456-426614174000",
    "intervalMs": 30000,
    "nextCheckpointAt": "2026-01-25T10:30:30.000Z"
  }
}
```

5.2 주기적 체크포인트 중지

```
POST /api/checkpoints/periodic/stop
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "123e4567-e89b-12d3-a456-426614174000"
}
```

5.3 주기적 체크포인트 상태 조회

```
GET /api/checkpoints/periodic/status/:agentId
```

응답 예시:

```
{
  "success": true,
  "status": {
    "isRunning": true,
    "intervalMs": 30000,
    "lastCheckpointAt": "2026-01-25T10:30:00.000Z",
    "nextCheckpointAt": "2026-01-25T10:30:30.000Z",
    "totalCheckpoints": 45
  }
}
```

---

6. 증분 체크포인트 (Incremental Checkpointing)

6.1 증분 체크포인트 생성

이전 체크포인트와의 차이만 저장하여 공간 절약.

```
POST /api/checkpoints/incremental
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "123e4567-e89b-12d3-a456-426614174000",
  "state": {
    "messages": [
      {
        "role": "user",
        "content": "New message"
      }
    ],
    "variables": {
      "counter": 6
    }
  },
  "baseCheckpointId": "ckpt-abc123"
}
```

응답 예시:

```
{
  "success": true,
  "checkpoint": {
    "checkpointId": "ckpt-inc789",
    "type": "incremental",
    "baseCheckpointId": "ckpt-abc123",
    "size": 128,
    "compressionRatio": 0.125
```

```
  }
}
```

6.2 증분 적용

기본 체크포인트에 증분을 적용하여 전체 상태 복원.

```
POST /api/checkpoints/incremental/apply
Content-Type: application/json
```

요청 본문:

```
{
  "baseCheckpointId": "ckpt-abc123",
  "incrementId": "ckpt-inc789"
}
```

응답 예시:

```
{
  "success": true,
  "mergedState": {
    "messages": [
      {
        "role": "user",
        "content": "Hello"
      },
      {
        "role": "assistant",
        "content": "Hi there!"
      },
      {
        "role": "user",
        "content": "New message"
      }
    ],
    "variables": {
      "counter": 6,
      "userName": "Alice"
    }
  }
}
```

---

7. 에러 응답 (Error Responses)

7.1 에러 형식

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request parameters",
    "details": {
      "field": "agentId",
      "issue": "must be a valid UUID"
    }
  }
}
```

7.2 에러 코드

코드	설명
VALIDATION_ERROR	요청 파라미터 검증 실패
AGENT_NOT_FOUND	에이전트를 찾을 수 없음
CHECKPOINT_NOT_FOUND	체크포인트를 찾을 수 없음
CHECKPOINT_CORRUPTED	체크포인트 데이터 손상
STATE_TOO_LARGE	상태 크기가 최대치 초과
INTEGRITY_CHECK_FAILED	무결성 검증 실패
SERIALIZATION_ERROR	직렬화 오류

---

8. 사용 예제 (Usage Examples)

8.1 Node.js 클라이언트

```
const axios = require('axios');

class CheckpointingClient {
  constructor(baseURL = 'http://localhost:3002/api') {
    this.client = axios.create({ baseURL });
  }

  // 체크포인트 생성
  async createCheckpoint(agentId, state, options = {}) {
    const response = await this.client.post('/checkpoints', {
      agentId,
      state,
      options: {
        type: 'full',
        ...options
      }
    });
    return response.data.checkpoint;
  }

  // 복구 실행
  async recover(agentId, checkpointId = null) {
    const response = await this.client.post('/checkpoints/recover', {
      agentId,
      checkpointId,
      verifyIntegrity: true,
      fallbackToLatest: true
    });
    return response.data.restoredState;
  }

  // 주기적 체크포인트 시작
  async startPeriodic(agentId, intervalMs = 30000) {
    const response = await this.client.post('/checkpoints/periodic/start', {
      agentId,
      intervalMs,
      idleCheckpointsEnabled: true,
      adaptiveInterval: true
```

```
    });
    return response.data;
  }

  // 체크포인트 목록 조회
  async listCheckpoints(agentId) {
    const response = await this.client.get(`/checkpoints/${agentId}`);
    return response.data.checkpoints;
  }
}

// 사용 예시
const client = new CheckpointingClient();

async function main() {
  try {
    const agentId = '123e4567-e89b-12d3-a456-426614174000';

    // 상태 정의
    const state = {
      messages: [
        { role: 'user', content: 'Hello' },
        { role: 'assistant', content: 'Hi there!' }
      ],
      variables: { counter: 5 },
      status: 'running'
    };

    // 체크포인트 생성
    const checkpoint = await client.createCheckpoint(agentId, state, {
      description: 'Initial checkpoint',
      tags: ['important']
    });
    console.log('Checkpoint created:', checkpoint.checkpointId);

    // 복구 테스트
    const restored = await client.recover(agentId, checkpoint.checkpointId);
    console.log('Restored state:', restored);

    // 주기적 체크포인트 시작
    await client.startPeriodic(agentId, 30000);
    console.log('Periodic checkpointing started');

  } catch (error) {
    console.error('Error:', error.response?.data || error.message);
  }
}

main();
```

8.2 Python 클라이언트

```
import requests
from typing import Optional, Dict, Any

class CheckpointingClient:
    def __init__(self, base_url: str = "http://localhost:3002/api"):
```

```python
        self.base_url = base_url

    def create_checkpoint(
        self,
        agent_id: str,
        state: Dict[str, Any],
        options: Optional[Dict[str, Any]] = None
    ) -> Dict[str, Any]:
        """체크포인트 생성"""
        response = requests.post(
            f"{self.base_url}/checkpoints",
            json={
                "agentId": agent_id,
                "state": state,
                "options": options or {"type": "full"}
            }
        )
        response.raise_for_status()
        return response.json()["checkpoint"]

    def recover(
        self,
        agent_id: str,
        checkpoint_id: Optional[str] = None
    ) -> Dict[str, Any]:
        """복구 실행"""
        response = requests.post(
            f"{self.base_url}/checkpoints/recover",
            json={
                "agentId": agent_id,
                "checkpointId": checkpoint_id,
                "verifyIntegrity": True,
                "fallbackToLatest": True
            }
        )
        response.raise_for_status()
        return response.json()["restoredState"]

    def start_periodic(
        self,
        agent_id: str,
        interval_ms: int = 30000
    ) -> Dict[str, Any]:
        """주기적 체크포인트 시작"""
        response = requests.post(
            f"{self.base_url}/checkpoints/periodic/start",
            json={
                "agentId": agent_id,
                "intervalMs": interval_ms,
                "idleCheckpointsEnabled": True,
                "adaptiveInterval": True
            }
        )
        response.raise_for_status()
        return response.json()
```

```python
# 사용 예시
client = CheckpointingClient()

try:
    agent_id = "123e4567-e89b-12d3-a456-426614174000"

    # 상태 정의
    state = {
        "messages": [
            {"role": "user", "content": "Hello"},
            {"role": "assistant", "content": "Hi there!"}
        ],
        "variables": {"counter": 5},
        "status": "running"
    }

    # 체크포인트 생성
    checkpoint = client.create_checkpoint(agent_id, state, {
        "description": "Initial checkpoint",
        "tags": ["important"]
    })
    print(f"Checkpoint created: {checkpoint['checkpointId']}")

    # 복구 테스트
    restored = client.recover(agent_id, checkpoint['checkpointId'])
    print(f"Restored state: {restored}")

    # 주기적 체크포인트 시작
    client.start_periodic(agent_id, 30000)
    print("Periodic checkpointing started")

except Exception as e:
    print(f"Error: {e}")
```

---

9. 성능 메트릭 (Performance Metrics)

9.1 작업 시간

작업	평균 시간	설명
상태 직렬화	1-3ms	JSON 변환
전체 체크포인트 생성	5-10ms	저장 포함
증분 체크포인트 생성	3-5ms	차이 계산 포함
복구	10-50ms	데이터베이스 조회 포함
무결성 검증	1-2ms	해시 검증

9.2 저장 공간

체크포인트 유형	평균 크기	압축 효과
전체 (Full)	1-5 MB	없음
증분 (Incremental)	100-500 KB	80-90% 감소
압축 (Compressed)	200-500 KB	60-70% 감소

---

10. 모범 사례 (Best Practices)

10.1 체크포인트 타이밍

적절한 체크포인트 지점: - 중요한 작업 완료 후 - 사용자 입력 처리 후 - 장기 계산 시작 전 - 상태 변경이 있을 때

```
// 좋은 예시
async function processTask(agentId, task) {
  await createCheckpoint(agentId, { status: 'started', task });

  const result = await executeTask(task);

  await createCheckpoint(agentId, { status: 'completed', result });
  return result;
}
```

10.2 상태 크기 최적화

```
// 나쁜 예시: 너무 큰 상태
const badState = {
  entireConversation: [...], // 수천 개의 메시지
  allVariables: {...}, // 수백 개의 변수
  largeBinaryData: buffer // 큰 바이너리 데이터
};

// 좋은 예시: 최적화된 상태
const goodState = {
  recentMessages: messages.slice(-10), // 최근 10개만
  essentialVariables: { counter, userId }, // 필수 변수만
  dataReference: s3Url // 큰 데이터는 참조만
};
```

10.3 에러 처리

```
try {
  const restored = await client.recover(agentId);
  // 복구 성공
} catch (error) {
  if (error.code === 'CHECKPOINT_CORRUPTED') {
    // 무결성 검증 실패: 백업 시도
    const backup = await client.recover(agentId, null);
    console.log('Recovered from backup');
  } else if (error.code === 'CHECKPOINT_NOT_FOUND') {
    // 체크포인트 없음: 초기 상태 사용
    console.log('No checkpoint found, using initial state');
  } else {
    throw error;
  }
}
```

---

11. OS 개념 매핑 (OS Concepts Mapping)

OS 개념	구현	설명
Process State	Agent State	에이전트의 실행 상태
Checkpoint	Checkpoint	특정 시점의 상태 스냅샷
Restore	Recovery	저장된 상태로 복원
Incremental Backup	Incremental Checkpoint	변경된 부분만 저장
Integrity Check	Hash Verification	데이터 무결성 검증

---

12. 변경 로그 (Changelog)

v1.0.0 (2025-01-25)
- 초기 릴리스
- 전체/증분 체크포인트 지원
- 주기적 자동 저장
- 무결성 검증
- 46/46 테스트 통과 (50.66% 커버리지)

문서 버전: 1.0.0 **최종 업데이트**: 2025-01-25 **연락처**: GitHub Issues